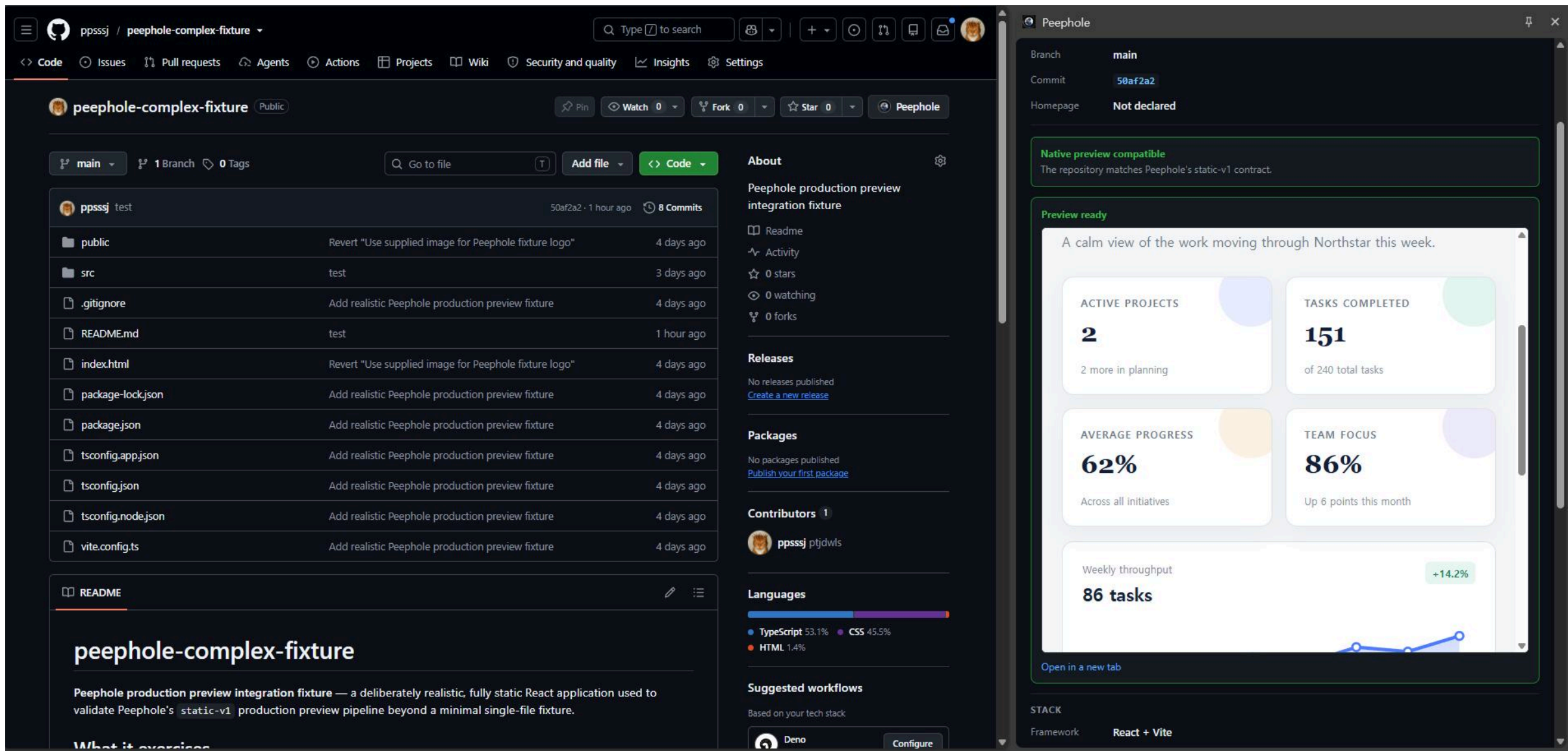




서비스

GitHub 저장소의 구조와 실행 가능 여부를 먼저 분석하고,
지원 가능한 정적 프론트엔드를 격리된 gVisor 환경에서 빌드해
Chrome Side Panel에서 바로 확인할 수 있는 Repository
Preview Platform v0.1.0 (서비스중)

[GitHub](#)

[Chrome Web Store](#)

기술 스택

TypeScript React WXT PostgreSQL
 Node.js AWS EC2 Linux gVisor
 Caddy

역할

Planning & Product Design

GitHub 저장소 Preview 문제 정의 및 사용자 흐름 설계
Repository 분석 및 Preview Eligibility 정책 설계

System & Infrastructure Development

Chrome Extension 및 Preview API 구현
PostgreSQL 기반 Preview Job Control Plane 설계
Linux/gVisor 기반 격리 Build Worker 구현
Artifact Host와 HTTPS Preview Delivery 구성
CPU·Memory·PID·Disk·Network 실행 제한 구현
Worker 장애 복구 및 Resource Reconciliation 설계
AWS EC2 Production 환경 구축 및 E2E 검증

작업 기간

MVP 개발 2026.08 - 2026.09

배포·운영·개선 2026.09 - 현재

구성원

1인 프로젝트

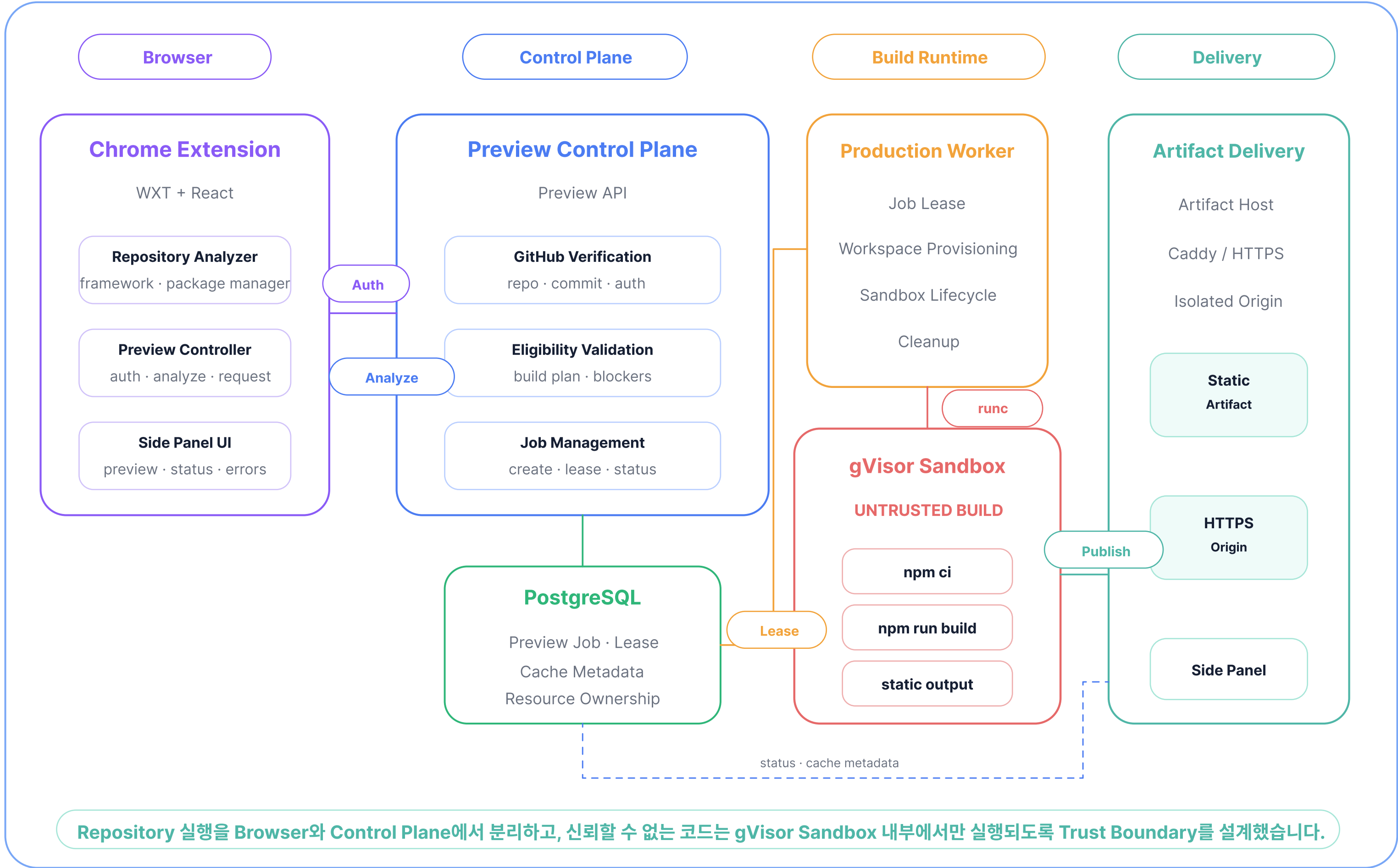
기획 UI/UX 설계 Chrome Extension Preview API
Database Linux Sandbox Infrastructure Security

Engineering Highlights

- GitHub branch tip이 아닌 exact commit 기준 Preview Pipeline 설계
- 실행 전 Framework·Package Manager·Build Plan·Blocker를 분석해 Preview Eligibility 판별
- Untrusted install/build process를 non-root gVisor Sandbox에서 격리 실행
- CPU·Memory·PID·Wall-clock 제한과 loop-backed ext4 Disk Hard Quota 적용
- Install/Build 단계별 Network Policy와 isolated HTTPS Artifact Origin 구성
- Worker 강제 종료 이후 runsc·mount·loop·network resource reconciliation 구현

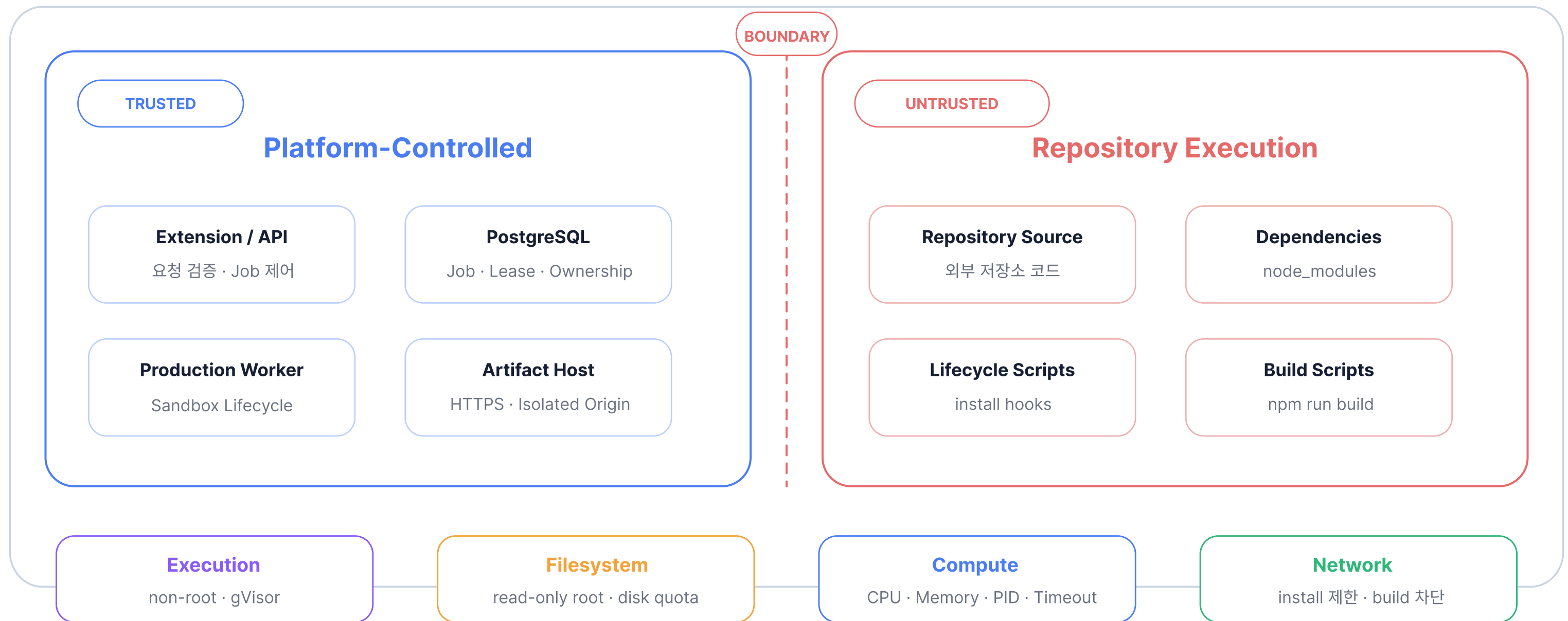
01. Repository Preview 시스템 아키텍처

저장소 분석·실행 제어·격리 빌드·결과 전달을 분리하고, 외부 Repository 코드는 Sandbox 내부에서만 실행되도록 설계했습니다.



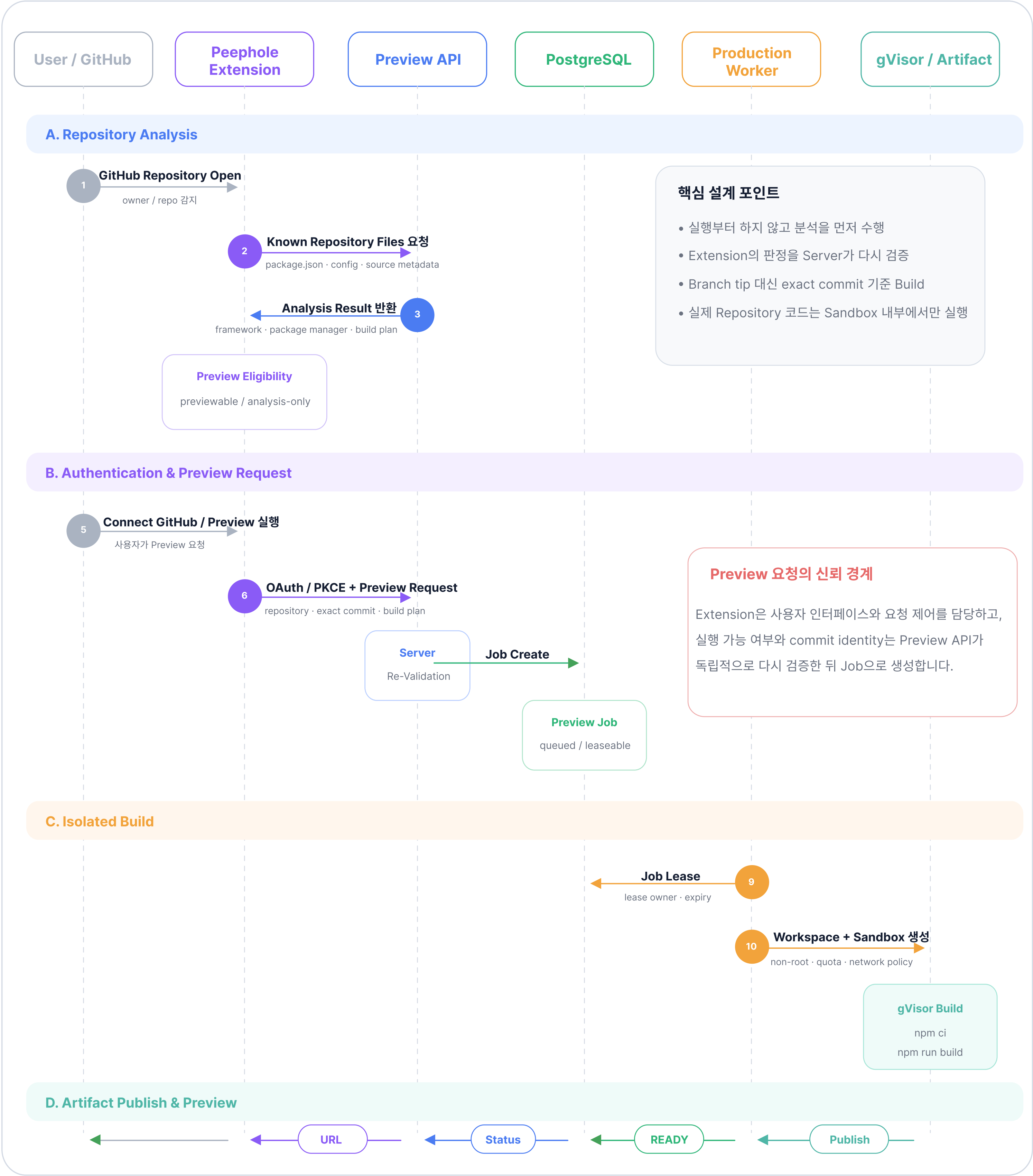
02. Trust Boundary와 실행 책임

지원 가능한 저장소도 신뢰하지 않는 입력으로 취급하고, 실행 권한·디스크·네트워크·리소스를 각각 독립적인 경계로 제한합니다.



03. Repository 분석부터 Preview 생성까지

실행 전에 Repository를 분석하고, 검증된 Preview 요청만 격리된 Build Worker로 전달해 HTTPS Artifact로 반환하는 End-to-End 흐름입니다.



04. 신뢰할 수 없는 Repository를 위한 Sandbox 설계

지원 가능한 저장소도 안전하다고 가정하지 않고, dependency lifecycle script와 build script까지 모두 신뢰하지 않는 실행으로 취급했습니다.

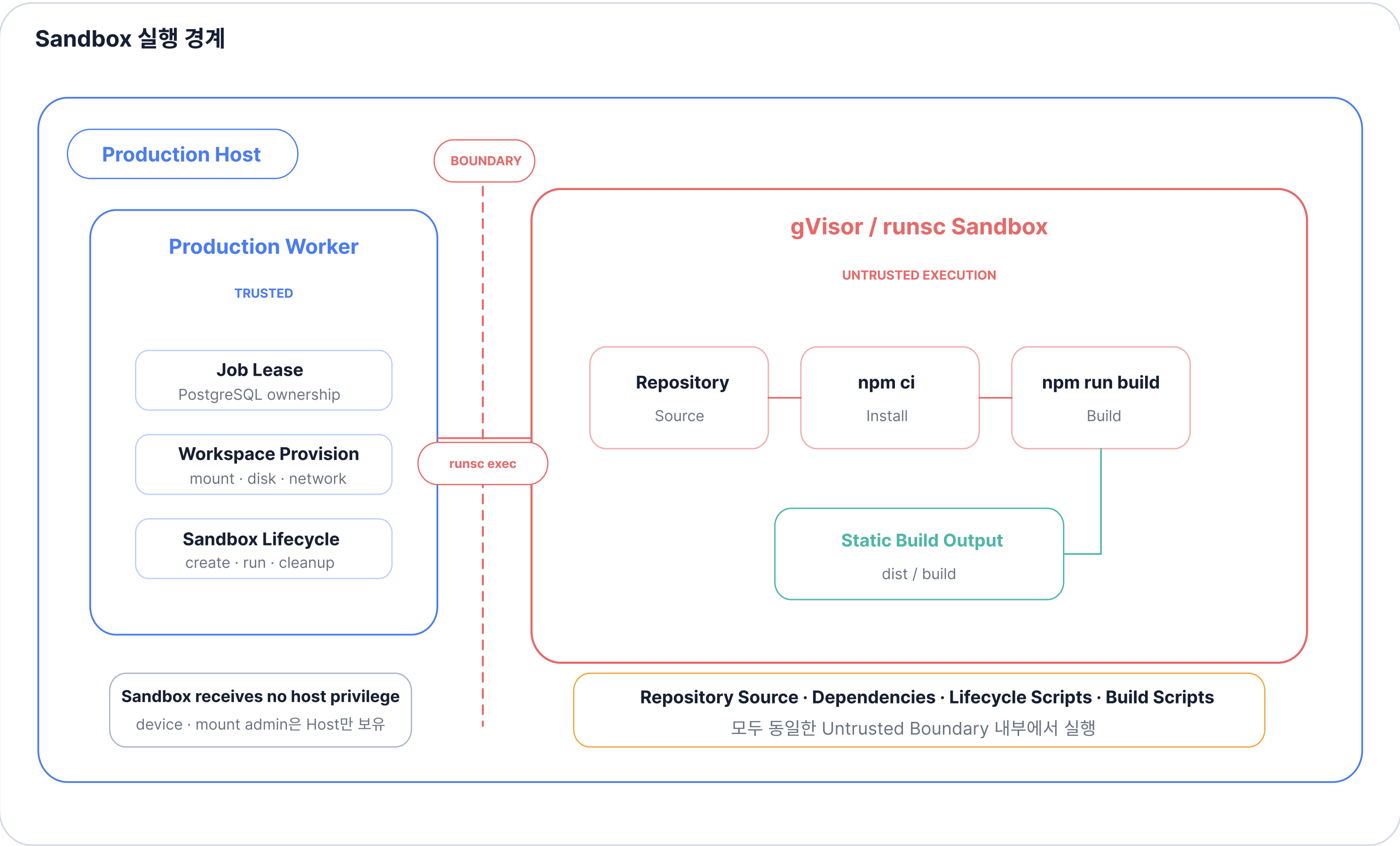
gVisor
real runsc sandbox

non-root
restricted execution

Disk Quota
loop-backed ext4

Network Policy
install / build 분리

Resource Limit
CPU · RAM · PID



실행 단계별 보안 제어

하나의 Sandbox만 두는 대신 실행·파일시스템·네트워크·리소스 경계를 각각 독립적으로 제한했습니다.

01. Execution

- real gVisor / runsc
- non-root user
- exact commit pinning
- host privilege 미전달

02. Filesystem

- guest root read-only
- loop-backed ext4 workspace
- hard capacity 적용
- job별 workspace 분리

03. Network

- Install 단계만 제한적 egress
- private / metadata 차단
- Build 단계 network disabled
- inter-job 접근 차단

04. Compute

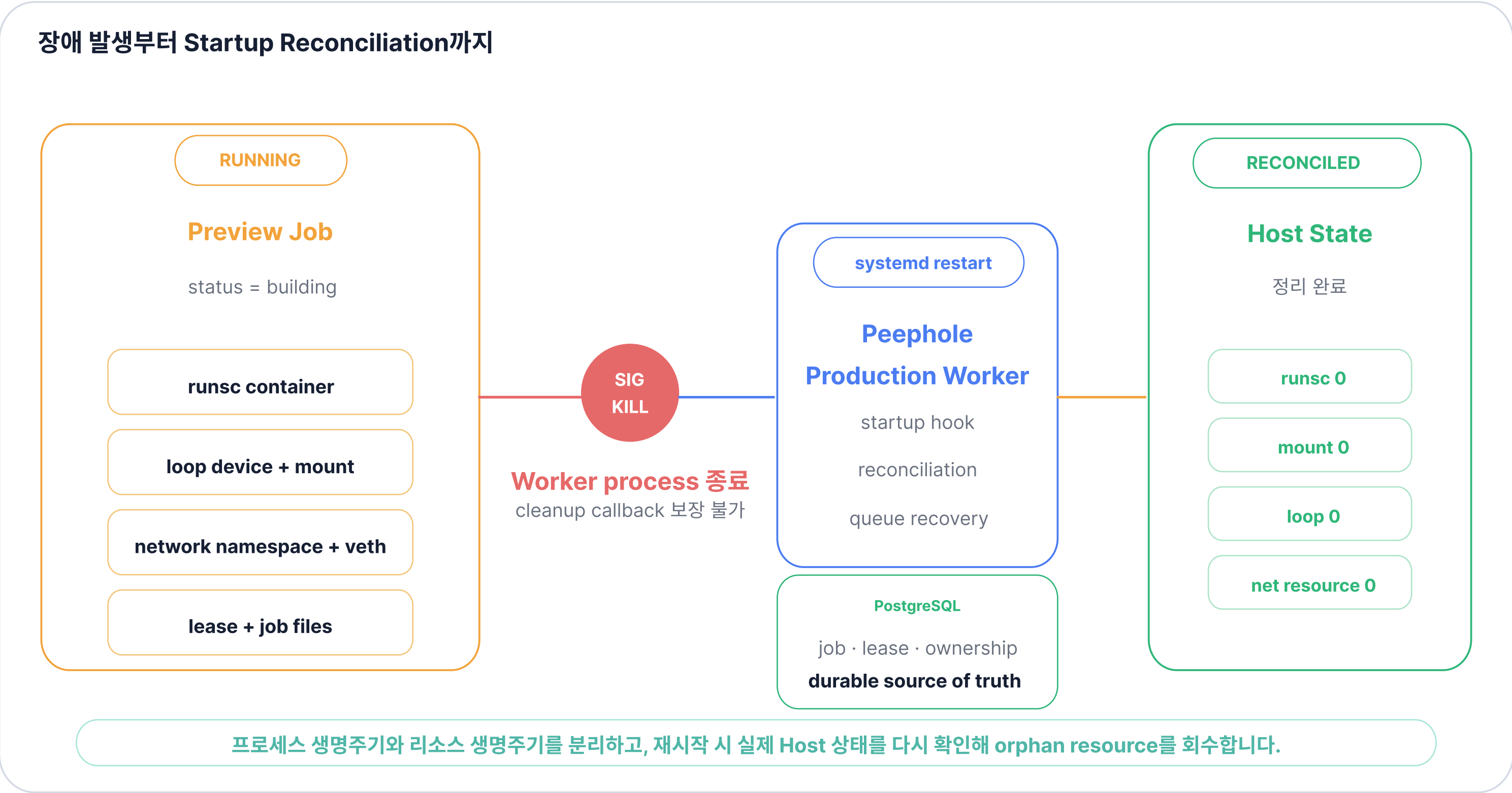
- CPU bound
- Memory limit
- PID / process limit
- wall-clock timeout

SUPPORTED ≠ TRUSTED

Preview 가능 여부와 코드의 신뢰 여부를 분리하고, 실행이 필요한 경우에만 제한된 Sandbox에서 처리했습니다.

05. Worker 장애 이후에도 남지 않는 실행 환경 설계

Job 상태만 복구하지 않고, Worker가 강제 종료된 이후 Host에 남을 수 있는 Sandbox·Disk·Network 자원까지 다시 탐색하고 정리하도록 설계했습니다.



Startup Reconciliation 흐름

메모리 상태를 신뢰하지 않고 PostgreSQL ownership과 Host의 실제 Linux 자원을 비교해 정리 여부를 판단합니다.



Reconciliation 대상

Job과 연결된 소유권을 지속 상태로 기록해, 장애 후에도 무엇을 회수해야 하는지 추적할 수 있도록 구성했습니다.



Recovery = Job 상태 복원 + 실제 Linux Resource Reconciliation